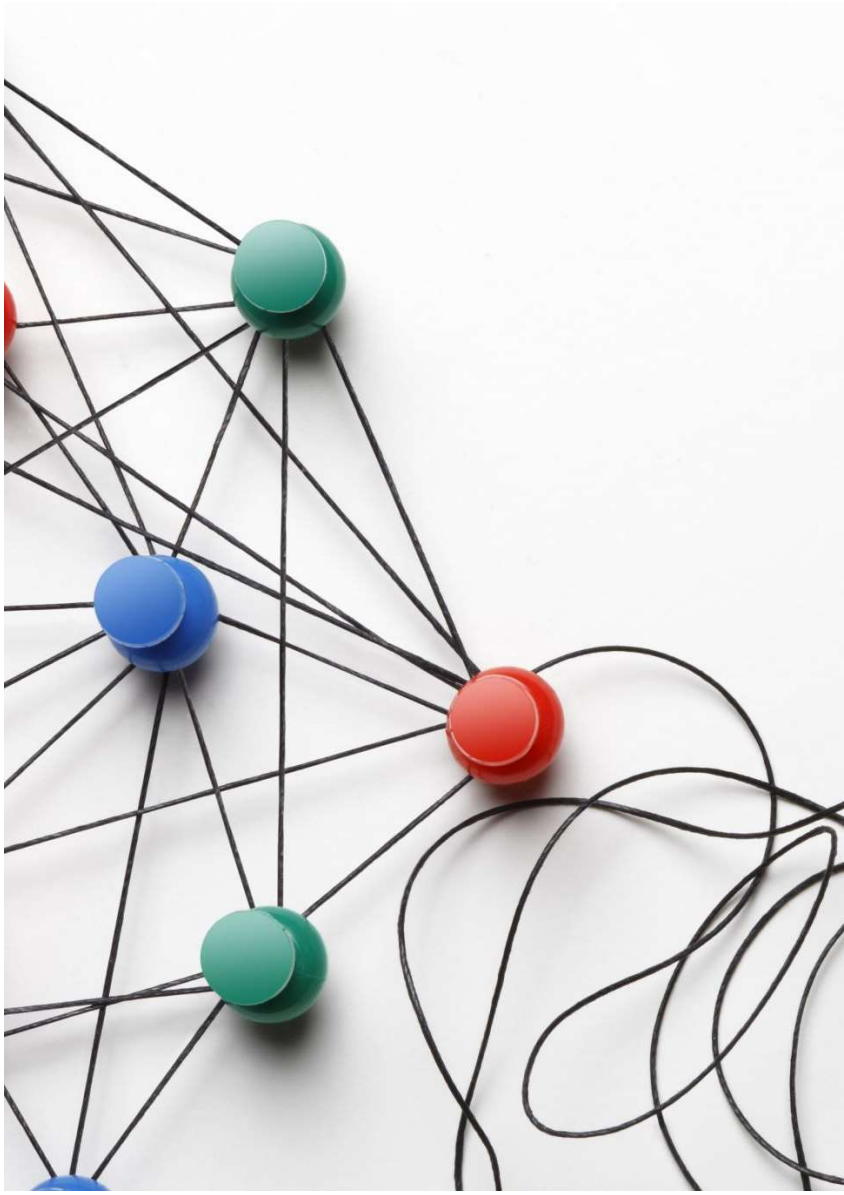

Computer Programming
Summer Interest Group
Session 18 / Concurrency 1

Romans 8:26-30

Likewise the Spirit helps us in our weakness. For we do not know what to pray for as we ought, but the Spirit himself intercedes for us with groanings too deep for words. And he who searches hearts knows what is the mind of the Spirit, because the Spirit intercedes for the saints according to the will of God. And we know that for those who love God all things work together for good for those who are called according to his purpose. For those whom he foreknew he also predestined to be conformed to the image of his Son, in order that he might be the firstborn among many brothers. And those whom he predestined he also called, and those whom he called he also justified, and those whom he justified he also glorified.

Objectives

- By the end of this session, you will be able to:
  - Explain what concurrency and threading are in Python.
  - Create and manage multiple threads using the threading module.
  - Synchronize access to shared data using locks.
  - Understand when threading is beneficial and when it’s limited by the GIL.



Concurrency

- Concurrency is the ability of a computer to perform multiple operations or tasks with apparent or real simultaneity.
- Apparent simultaneity results from rapidly shifting context between processes
  - This can be done on hosts with any number of processing cores (1 or more)
- Real simultaneity results from running different operations on different processing cores
  - This requires the host have at least as many processing cores as there are concurrent operations being performed
- Modern operating systems typically implement a hybrid approach where operations are spread across available cores.
  - Multiple operations may be shared on a single processing core

Concurrency

Benefits

- Ability to use computing resource more effectively
- Ability to continue computation while waiting for other operations to complete

Risks

- Harder to implement
- More error prone
- Types of errors are hard to reproduce and resolve

Python Concurrency Approaches

- **Multiprocessing**
  - Multiple Python runtimes running concurrently
  - No shared memory
  - Coordination accomplished through inter-process communications
  - Implements real concurrency
- **Multithreading**
  - Single Python runtime
  - Memory shared across threads
  - Coordination options are much more varied
  - Implements apparent concurrency
- **Asynchronous I/O**
  - Single Python runtime
  - Allows Input / Output operations to be performed without blocking
  - Requires use of additional Python keywords
  - Implements apparent concurrency

Python Threading Model

- Each Python runtime has a single “Global Interpreter Lock” (GIL)
- This operates in a single process and switches rapidly between all threads within the same process
- This gives apparent concurrency
- Other programming languages (C, C++, Java, Go,…) implement threading to enable real concurrency
- Python predates widespread adoption of multi-core commodity CPU’s so did not implement real multi-threaded concurrency
- There is a push to eliminate the GIL and transition to real concurrency, but it has not yet been fully implemented
    - Why Python 3.14 GIL Update is Significant for Threading

Threading Bottom Line

- The standard Python approach to threading is to implement apparent concurrency within a single thread.
- The GIL manages Python thread execution within that single processing thread
- Newer implementation of Python are migrating away from this approach (i.e. Free Threads)
- But it’s the simplest way Python provides to implement concurrency

Simple example

- The function **`wait`** is run in a separate thread
- Note the main thread will finish before the thread running **`wait`** finishes.
- We create a **`threading.Thread`** object with the **`target`** attribute being the function we want to run.
- The thread does not start running until the thread's start method is called.
- Once started it will keep running until completed even if the parent thread ends

```
import threading
import time

def wait():
    print(“Wait thread - begin")
    time.sleep(1.0)
    print(“Wait thread - end")

thread = threading.Thread(target=wait)
print("Main thread - begin")
thread.start()
print("Main thread - end")
```

Daemon thread

- Pronounced “day’-mon”
- The life of a daemon thread does not exceed the life of its parent thread or process.
  - It may terminate earlier than the parent, but never later
- In this case, the **`wait`** function will not complete because the parent thread terminates before the daemon thread completes.
- Use **`daemon = True`** when creating the **`Thread`** instance

```
import threading
import time

def wait():
    print(“Wait thread - begin")
    time.sleep(1.0)
    print(“Wait thread - end")

thread = threading.Thread(
    target=wait,
    daemon=True)
print("Main thread - begin")
thread.start()
print("Main thread - end")
```

Locks

- Race conditions occur when multiple threads are attempting to access the same resource at the same time
- Thread locks restrict processing of locked code segments so no more than one of them can be executed at any time
- The **`global`** label in code to right designates counter as being a shared resource available to all the threads
  - Use of global is not a good idea, but I'm using it here to illustrate a point
- We do not want multiple threads to simultaneously read or modify **`counter`** as this could lead to unexpected results.
- Access to it is protected with the with lock context manager
- Only one thread at a time is allowed to run the locked code block
- Note how this code works in practice when running it

```
import threading
import time

lock = threading.Lock()
counter: int = 1

def wait():
    global counter
    with lock:
        my_counter = counter
        counter += 1

    print(f"Wait - begin {my_counter}")
    time.sleep(0.5)
    print(f"Wait - end {my_counter}")

def main():
    threads = [
        threading.Thread(target=wait)
        for _ in range(10)]
    print("Main thread - begin")
    for thread in threads:
        thread.start()
    print("Main thread - end")
```

A note on use of locks

Use of thread locks may result something called a “deadlock”

A deadlock occurs when different threads are blocked from progressing because they require a resource which another thread has locked

This is similar to a circular dependency that cannot be resolved.

Usually, resolution of a deadlock condition requires a forced termination of the software and refactoring the system to remove deadlock situations

Thread class

- A class can be built from the base Thread class
- This allows additional local context to be associated with the thread
  - Refer to earlier comment about relying on global variables
- Two requirements:
  - The Thread __init__ method must be called. If the derived class has its own __init__ method, it must explicitly call the Thread __init__ method as shown
- It must hace a run method defined and which takes no arguments (other than self) and returns no values
  - This method is what will run when the thread is active

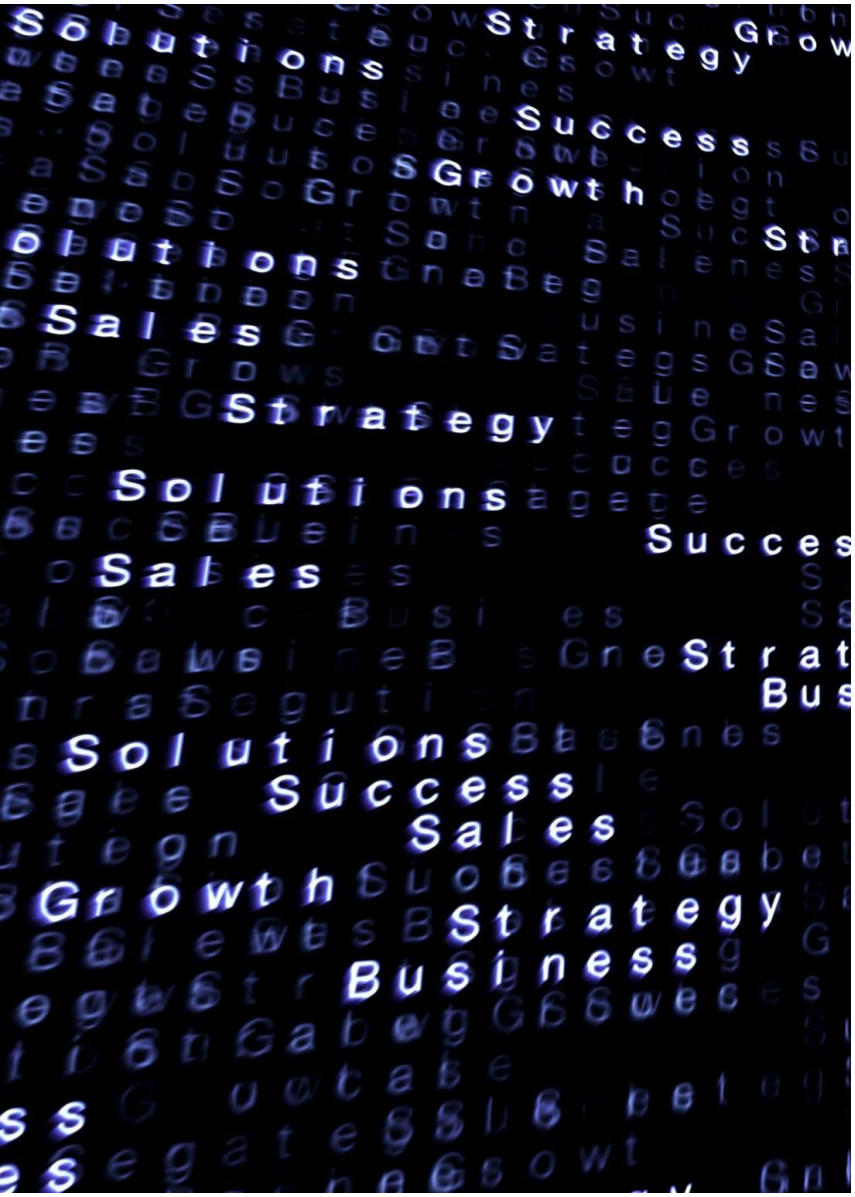
```
import threading
import time

class MyThread(threading.Thread):
    counter: int = 1

    def __init__(self, wait: float):
        super().__init__()
        self._wait = wait
        self._counter = MyThread.counter
        MyThread.counter += 1

    def run(self):
        print(f"Wait thread - begin {self._counter}")
        time.sleep(self._wait)
        print(f"Wait thread - end {self._counter}")

threads = [MyThread(wait=0.5) for _ in range(10)]
print("Main thread - begin")
for thread in threads:
    thread.start()
print("Main thread - end")
```



Activities

- A thread pool is a collection of thread objects that can be used to perform tasks asynchronously (concurrently)
- Thread pools have a maximum number of threads they can run at any given time.
- Write a thread pool class that is initialized with the maximum number of threads it will run.
- Add tasks by giving the thread pool a function to run in a thread and pass its arguments as *args or **kwargs.
- Add the function and its arguments to a queue and run them in the order received, each in its own thread.
- Use the is_alive method in the Thread class to determine when a thread has completed.